

Auditeur de Cohérence Médicale

Préparation à la soutenance - solution d'IA, déploiement et pilotage

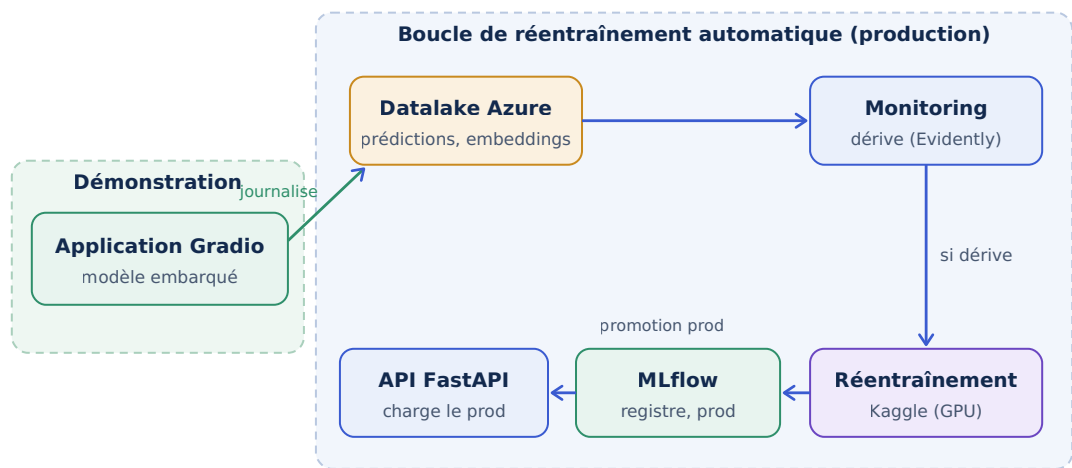
Cette fiche récapitule ma solution, le déroulé de ma présentation, les compétences démontrées, et les réponses aux questions probables. Elle me sert de support de révision.

1. Le besoin (cahier des charges)

Dans un service de radiologie, une radiographie thoracique peut être associée par erreur au compte rendu d'un autre patient. Une telle incohérence est un risque pour la sécurité des soins. La solution reçoit une paire image-texte et estime la probabilité qu'elles décrivent le même examen, afin de signaler les associations incohérentes pour une revue humaine. Elle ne pose pas de diagnostic : c'est un outil d'audit multimodal.

Traduction technique : un problème de classification binaire. Pour une paire (image, compte rendu), prédire cohérent (1) ou incohérent (0), avec un score de confiance.

2. La solution en un coup d'oeil



La boucle de production est automatique : le datalake alimente le monitoring, qui déclenche un réentraînement sur dérive ; le nouveau modèle est promu dans MLflow, et l'API sert toujours le modèle de production. L'application de démonstration embarque son propre modèle : elle alimente le datalake mais n'est pas reliée à l'API. C'est un choix assumé (démonstration autonome contre composant de production).

3. Déroulé de la présentation (5 minutes)

Temps	Contenu
0:00 - 0:45	Le besoin métier et la traduction en problème de classification binaire.
0:45 - 1:45	Le modèle : BioViL-T (encodeurs image et texte spécialisés radiologie) plus un classifieur à attention croisée. Résultat : environ 89 % d'exactitude.
1:45 - 3:15	La chaîne MLOps (schéma) : entraînement, MLflow, registre, promotion, API, monitoring, réentraînement automatique.
3:15 - 4:15	

	Démonstration vidéo : un cas cohérent (91,8 %) et un cas incohérent, pour montrer que le système distingue le vrai du faux.
4:15 - 5:00	La boucle automatique et la promotion conditionnelle. Choix d'architecture assumés.

4. Les compétences démontrées

Compétence évaluée	Ce que je montre
Rédiger un cahier des charges pour la solution d'IA	Le besoin d'audit de cohérence radio / compte rendu, traduit en classification binaire.
Créer un algorithme d'IA adapté aux données	Classifieur à attention croisée sur BioViL-T ; réglages repris de mon entraînement (batch 64, 100 époques max, arrêt anticipé).
Adapter l'infrastructure via des API pour la production	API FastAPI (/health, /predict, /reload-model), conteneurisée Docker, qui charge le modèle de production depuis MLflow.
Concevoir des pipelines d'intégration et déploiement continu	Workflows GitHub Actions : tests (CI), déploiement de l'application, monitoring planifié, réentraînement.
Développer des scripts de réentraînement automatisé	Boucle complète : dérive détectée, signal, entraînement sur Kaggle, enregistrement au registre, promotion conditionnelle.
Piloter la performance via un outil de monitoring	Monitoring de dérive sémantique avec Evidently sur les embeddings de texte, seuil de dérive à 0,55.

5. Mon point fort à mettre en avant

La promotion conditionnelle automatique. Beaucoup de chaînes MLOps s'arrêtent à une promotion manuelle : après un réentraînement, un humain décide de mettre le nouveau modèle en production. Ma chaîne va plus loin : la promotion est automatique et conditionnelle. Le nouveau modèle ne devient "prod" que si sa perte de validation est meilleure que celle du modèle en place. Cette comparaison protège la production d'un modèle dégradé, sans intervention humaine.

Preuve en conditions réelles : lors d'un réentraînement, un modèle de perte 0,30 a été correctement écarté car la production était meilleure (0,28). Le système a gardé l'ancien modèle. C'est la démonstration que la protection fonctionne.

6. Chiffres et faits clés à retenir

- Jeu d'entraînement : 20 000 paires équilibrées (10 000 cohérentes, 10 000 incohérentes), issues de CheXpert.
- Performance : environ 89 % d'exactitude de validation ; perte de validation du modèle de production autour de 0,28.
- Seuil de décision : 0,5. Seuil de dérive qui déclenche le réentraînement : 0,55.
- Démonstration : cas cohérent à 91,8 % de confiance ; cas incohérent correctement écarté.
- Registre MLflow : plusieurs versions ; la version de production porte l'alias "prod".

7. Questions probables et réponses

Pourquoi l'application et l'API ne sont-elles pas reliées ?

Ce sont deux composants aux rôles distincts. L'application est une démonstration autonome : elle embarque son propre modèle et réalise l'inférence localement, sans dépendre d'une API en ligne en permanence. L'API, elle, est le composant de production relié au registre MLflow : c'est elle qui sert toujours le modèle à jour. Maintenir une API en ligne en continu aurait consommé mon budget cloud, d'où le choix d'une application autonome pour la démonstration.

Comment fonctionne la promotion d'un modèle en production ?

Après chaque entraînement, le modèle est enregistré au registre MLflow avec un numéro de version. Je compare sa perte de validation à celle du modèle en production. Il ne reçoit l'alias "prod" que s'il est meilleur. Cette promotion conditionnelle est automatique, et je l'ai vérifiée : un réentraînement moins bon a été correctement écarté.

Que fait la route /reload-model de l'API ?

Elle recharge le modèle de production depuis MLflow à chaud, sans redémarrer l'API. Après un réentraînement qui promeut un nouveau modèle, un appel à cette route suffit pour que l'API serve la nouvelle version.

Pourquoi un repli local dans l'API ?

Par résilience. L'API charge le modèle de production depuis MLflow, la source de vérité. Si le serveur MLflow est momentanément indisponible, l'API sert la dernière version validée en local plutôt que de tomber en panne. La copie locale est rafraîchie à chaque chargement réussi, donc elle reste à jour.

Comment le réentraînement se déclenche-t-il automatiquement ?

Le monitoring mesure chaque jour la dérive sémantique avec Evidently. Au-delà du seuil de 0,55, il envoie un signal (repository_dispatch) à mon dépôt. Ce signal déclenche un workflow qui relance l'entraînement sur Kaggle via son API.

Pourquoi Kaggle pour l'entraînement, et pas le cloud de production ?

Parce que l'entraînement exige un GPU, et que mon compte cloud n'y donne pas accès (quota GPU verrouillé). J'ai déporté le calcul sur Kaggle, qui offre un GPU gratuit et une API pour lancer l'entraînement à distance. La structure de la chaîne reste identique ; seule la ressource de calcul change.

Qu'est-ce que la dérive, et comment la mesurez-vous ?

La dérive est un changement dans la distribution des données de production par rapport à la référence. Je la mesure sur les 128 dimensions de l'embedding sémantique des comptes rendus : au-delà d'une part de 0,55 de dimensions en dérive, je déclare une dérive et je déclenche un réentraînement.

Comment garantisiez-vous la traçabilité et le retour arrière ?

Chaque entraînement crée une version numérotée dans le registre MLflow, avec ses paramètres et métriques. Pour revenir à une version antérieure, il suffit de déplacer l'alias "prod" sur cette version. Le versionnage et le retour arrière sont donc intégrés.

Votre serveur MLflow est-il sécurisé ?

Il n'a pas d'authentification à ce stade, ce que j'assume : son adresse est difficile à deviner et il ne contient pas de données patients. En production, je le sécuriserais avec l'authentification native de MLflow ou une passerelle d'identité du cloud, et je rangerais les identifiants comme secrets. J'ai identifié la limite et sa solution.

Pourquoi les comptes rendus sont-ils en anglais ?

Parce que le modèle repose sur BioViL-T, entraîné sur des données radiologiques anglophones (CheXpert). C'est la langue de référence du domaine en imagerie médicale.

8. Points à assumer clairement

Trois choix sont des décisions lucides sous contrainte, à présenter comme telles :

- **Kaggle au lieu d'un GPU cloud** : contrainte de budget et de quota, pas un défaut de conception.
- **Application autonome (démon) plus API reliée à MLflow (production)** : deux composants aux rôles distincts.
- **Serveur MLflow sans authentification** : limite identifiée, solution connue, non bloquante pour la démonstration.

9. Questions pour m'auto-évaluer

À me poser à voix haute, sans regarder les réponses de la section 7.

- Quelle est la différence entre l'entraînement et l'inférence dans ma solution ?
- Que se passe-t-il, étape par étape, entre la détection d'une dérive et la mise en production d'un nouveau modèle ?
- Pourquoi un modèle réentraîné n'est-il pas toujours promu en production ?
- Quels sont mes deux composants d'inférence, et lequel est relié à MLflow ?
- Sur quelle mesure repose ma décision de promotion, et pourquoi celle-là ?
- Quelle est la seule vraie limite de ma chaîne, et comment la corrigerais-je ?